

PwC — Customer Segmentation & Persona Automation

Objective

Build an end-to-end customer segmentation solution from event-level e-commerce data, create interpretable customer personas, and provide an automated way to assign a customer to a persona.

Data integrity rule: The original `CustomerSegmentation.db` is read-only. Exact duplicates are removed only from an in-memory working copy after approval; the original database is never overwritten.

1. Business Understanding

Business problem: The business wants to understand customer behaviour and group customers into meaningful segments that can support targeted marketing.

Key questions

- Which customers are casual browsers?
- Which customers show strong engagement?
- Which customers add products to cart but do not purchase?
- Which customers have purchased?
- Can we automatically assign new customer behaviour to a persona?

Success criteria

1. Create customer-level behavioural features.
2. Select a defensible clustering solution.
3. Profile and name the clusters using business behaviour.
4. Produce a customer-to-persona mapping.
5. Provide an automation function for future customer records.

Load DataSets

```
from google.colab import files
uploaded = files.upload()
<IPython.core.display.HTML object>
Saving CustomerSegmentation.db to CustomerSegmentation.db

import sqlite3
import pandas as pd
```

```

import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, calinski_harabasz_score,
davies_bouldin_score

DB_PATH = "CustomerSegmentation.db"

con = sqlite3.connect(DB_PATH)
raw = pd.read_sql("SELECT * FROM sample_table", con)
con.close()

print("Original shape:", raw.shape)
print(raw.head())

```

Original shape: (6000, 9)

	event_time	event_type	product_id	category_id	\
0	2019-10-09	view	26020347	2053013562837697343	
1	2019-10-20	view	1005217	2053013555631882655	
2	2019-10-26	purchase	1004161	2053013555631882655	
3	2019-10-11	view	7201811	2053013553165631753	
4	2019-10-30	view	1004708	2053013555631882655	

	category_code	brand	price	user_id	\
0	None	None	8.96	530851738	
1	electronics.smartphone	xiaomi	158.31	562301163	
2	electronics.smartphone	xiaomi	82.34	557935258	
3	None	lego	16.71	538874350	
4	electronics.smartphone	huawei	152.15	565583963	

	user_session
0	63ac3d69-5f2e-4cf6-92e4-01fa067c9c4c
1	07378124-bb89-4345-9c72-0ba152e012f1
2	11542b7b-dea0-4c09-876f-13997f8d58ec
3	0cc10db1-87a1-4833-a391-572cc9cdb915
4	b2eefc6c-03b8-4ed9-8e66-bb50c17d7908

2. Data Quality — inspect first, do not modify the source

The original database is never changed. We only inspect it and then create a separate working DataFrame.

```

print("Shape:", raw.shape)
print("\nData types:")
print(raw.dtypes)

```

```
print("\nMissing values:")
print(raw.isna().sum())

print("\nEvent distribution:")
print(raw["event_type"].value_counts())

print("\nExact duplicate rows:", raw.duplicated().sum())
print("\nUnique customers:", raw["user_id"].nunique())
```

Shape: (6000, 9)

Data types:

event_time	object
event_type	object
product_id	int64
category_id	int64
category_code	object
brand	object
price	float64
user_id	int64
user_session	object
dtype:	object

Missing values:

event_time	0
event_type	0
product_id	0
category_id	0
category_code	1975
brand	851
price	0
user_id	0
user_session	0
dtype:	int64

Event distribution:

event_type	
view	5763
cart	132
purchase	105
Name: count, dtype:	int64

Exact duplicate rows: 2

Unique customers: 5961

Duplicate handling

The dataset contains 2 exact duplicate rows. Because duplicates can slightly inflate engagement counts, they are removed only from the **working copy**.

The original `.db` file remains unchanged.

```
work = raw.drop_duplicates().copy()

print("Working shape after removing exact duplicates:", work.shape)
print("Original database rows still unchanged in `raw`:", len(raw))
```

```
Working shape after removing exact duplicates: (5998, 9)
Original database rows still unchanged in `raw`: 6000
```

3. Feature Engineering — convert event-level data to customer-level data

One row in the source is an event, not necessarily a customer.

We therefore aggregate by `user_id`.

Features:

- `total_events`: total recorded activities
- `views`: number of view events
- `carts`: number of cart events
- `purchases`: number of purchase events
- `sessions`: distinct sessions
- `unique_products`: distinct products interacted with
- `unique_categories`: distinct categories interacted with
- `unique_brands`: distinct known brands interacted with
- `total_spend`: total purchase value
- `avg_purchase_value`: average purchase value
- `active_days`: number of days with activity
- `recency_days`: days since last observed activity
- `cart_rate`: carts / total events
- `purchase_rate`: purchases / total events

```
work["event_time"] = pd.to_datetime(work["event_time"])

g = work.groupby("user_id")

customer = pd.DataFrame(index=g.size().index)
customer["total_events"] = g.size()
customer["views"] = g["event_type"].apply(lambda s: (s ==
"view").sum())
customer["carts"] = g["event_type"].apply(lambda s: (s ==
```

```

"cart").sum())
customer["purchases"] = g["event_type"].apply(lambda s: (s ==
"purchase").sum())
customer["sessions"] = g["user_session"].nunique()
customer["unique_products"] = g["product_id"].nunique()
customer["unique_categories"] = g["category_id"].nunique()
customer["unique_brands"] = g["brand"].nunique()

purchase_only = work[work["event_type"].eq("purchase")]
spend = purchase_only.groupby("user_id")["price"].agg(["sum",
"mean"]).rename(
    columns={"sum": "total_spend", "mean": "avg_purchase_value"}
)
customer = customer.join(spend).fillna({"total_spend": 0,
"avg_purchase_value": 0})

customer["active_days"] = g["event_time"].apply(lambda s:
s.dt.date.nunique())
last_activity = g["event_time"].max()
analysis_date = work["event_time"].max() + pd.Timedelta(days=1)
customer["recency_days"] = (analysis_date - last_activity).dt.days

customer["cart_rate"] = customer["carts"] / customer["total_events"]
customer["purchase_rate"] = customer["purchases"] /
customer["total_events"]

print(customer.shape)
display(customer.head())

```

(5961, 14)

```

{"summary": "{\n  \"name\": \"display(customer\", \n  \"rows\": 5, \n  \"fields\": [\n    {\n      \"column\": \"user_id\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 24901271, \n        \"min\": 388594905, \n        \"max\": 449749968, \n        \"num_unique_values\": 5, \n        \"samples\": [\n          408030870, \n          449749968, \n          419533799\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }\n    }, \n    {\n      \"column\": \"total_events\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0, \n        \"min\": 1, \n        \"max\": 1, \n        \"num_unique_values\": 1, \n        \"samples\": [\n          1\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }\n    }, \n    {\n      \"column\": \"views\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0, \n        \"min\": 1, \n        \"max\": 1, \n        \"num_unique_values\": 1, \n        \"samples\": [\n          1\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }\n    }, \n    {\n      \"column\": \"carts\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0, \n        \"min\": 0, \n

```

```

{"max": 0, "num_unique_values": 1, "samples": 1, "semantic_type": "categorical", "description": "unique_products", "column": "unique_products", "properties": {"dtype": "int"}, "std": 0, "min": 1, "max": 1, "num_unique_values": 1, "samples": 1, "semantic_type": "categorical", "description": "unique_categories", "column": "unique_categories", "properties": {"dtype": "int"}, "std": 0, "min": 1, "max": 1, "num_unique_values": 1, "samples": 1, "semantic_type": "categorical", "description": "unique_brands", "column": "unique_brands", "properties": {"dtype": "int"}, "std": 0, "min": 0, "max": 1, "num_unique_values": 2, "samples": 1, "semantic_type": "categorical", "description": "total_spend", "column": "total_spend", "properties": {"dtype": "float"}, "std": 0.0, "min": 0.0, "max": 0.0, "num_unique_values": 1, "samples": 1, "semantic_type": "categorical", "description": "avg_purchase_value", "column": "avg_purchase_value", "properties": {"dtype": "float"}, "std": 0.0, "min": 0.0, "max": 0.0, "num_unique_values": 1, "samples": 1, "semantic_type": "categorical", "description": "active_days", "column": "active_days", "properties": {"dtype": "int"}, "std": 0, "min": 1, "max": 1, "num_unique_values": 1, "samples": 1, "semantic_type": "categorical", "description": "recency_days", "column": "recency_days", "properties": {"dtype": "int"}, "std": 8, "min": 4, "max": 26, "num_unique_values": 4, "samples": 10, "semantic_type": "categorical", "description": "cart_rate", "column": "cart_rate", "properties": {"dtype": "float"}

```

```

\ "number\"," ,\n          \ "std\":" : 0.0,\n          \ "min\":" : 0.0,\n
\ "max\":" : 0.0,\n          \ "num_unique_values\":" : 1,\n          \ "samples\":" :
[\n          0.0\n          ],\n          \ "semantic_type\":" : \ "\",\n
\ "description\":" : \ "\",\n          }\n          },\n          {\n          \ "column\":" :
\ "purchase_rate\"," ,\n          \ "properties\":" : {\n          \ "dtype\":" :
\ "number\"," ,\n          \ "std\":" : 0.0,\n          \ "min\":" : 0.0,\n
\ "max\":" : 0.0,\n          \ "num_unique_values\":" : 1,\n          \ "samples\":" :
[\n          0.0\n          ],\n          \ "semantic_type\":" : \ "\",\n
\ "description\":" : \ "\",\n          }\n          }\n          ]\n          }","type":"dataframe"}

```

4. Exploratory Data Analysis

The source is very sparse: most customers have one recorded event. Therefore, the segmentation is a behavioural snapshot of the observed month rather than a long-term lifetime-value segmentation.

We inspect distributions and event behaviour before modelling.

```
print(customer.describe().T)
```

```

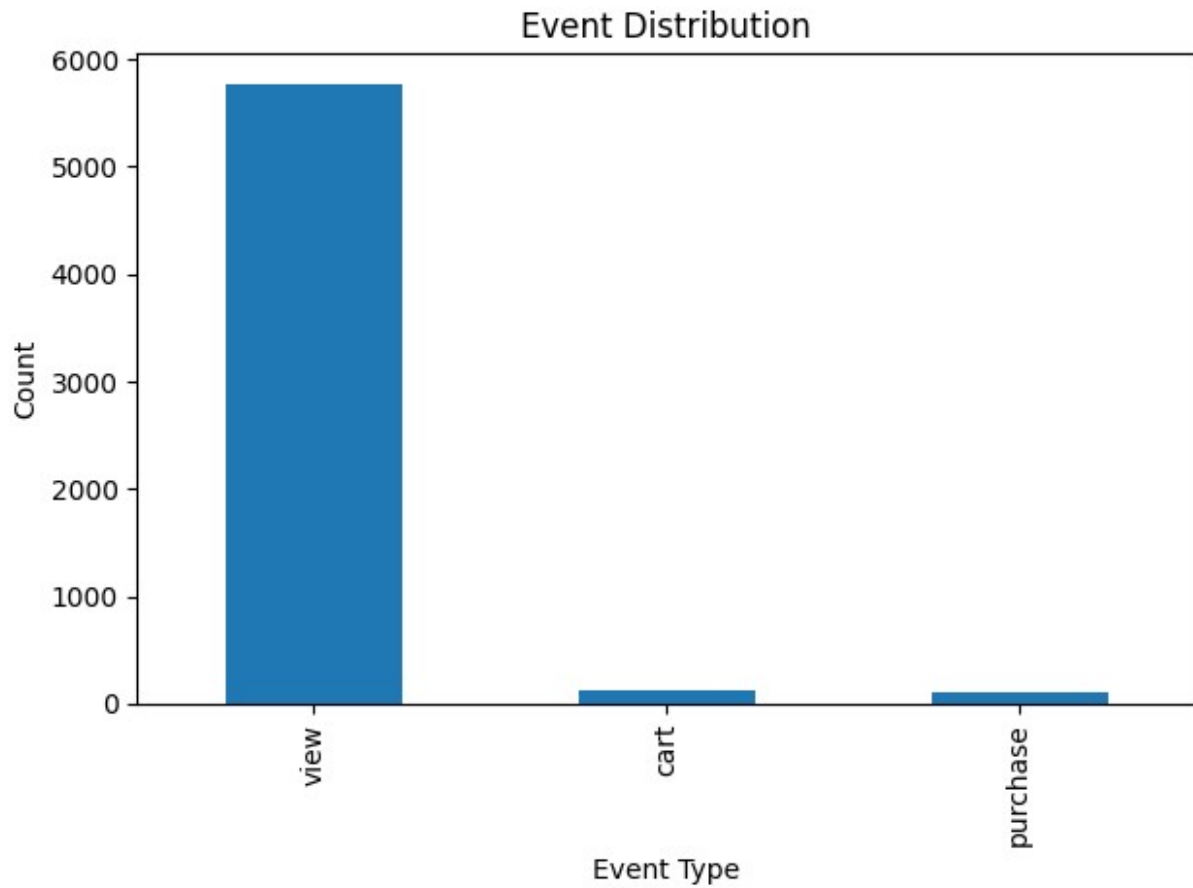
event_counts = work["event_type"].value_counts()
event_counts.plot(kind="bar", title="Event Distribution")
plt.xlabel("Event Type")
plt.ylabel("Count")
plt.tight_layout()
plt.show()

```

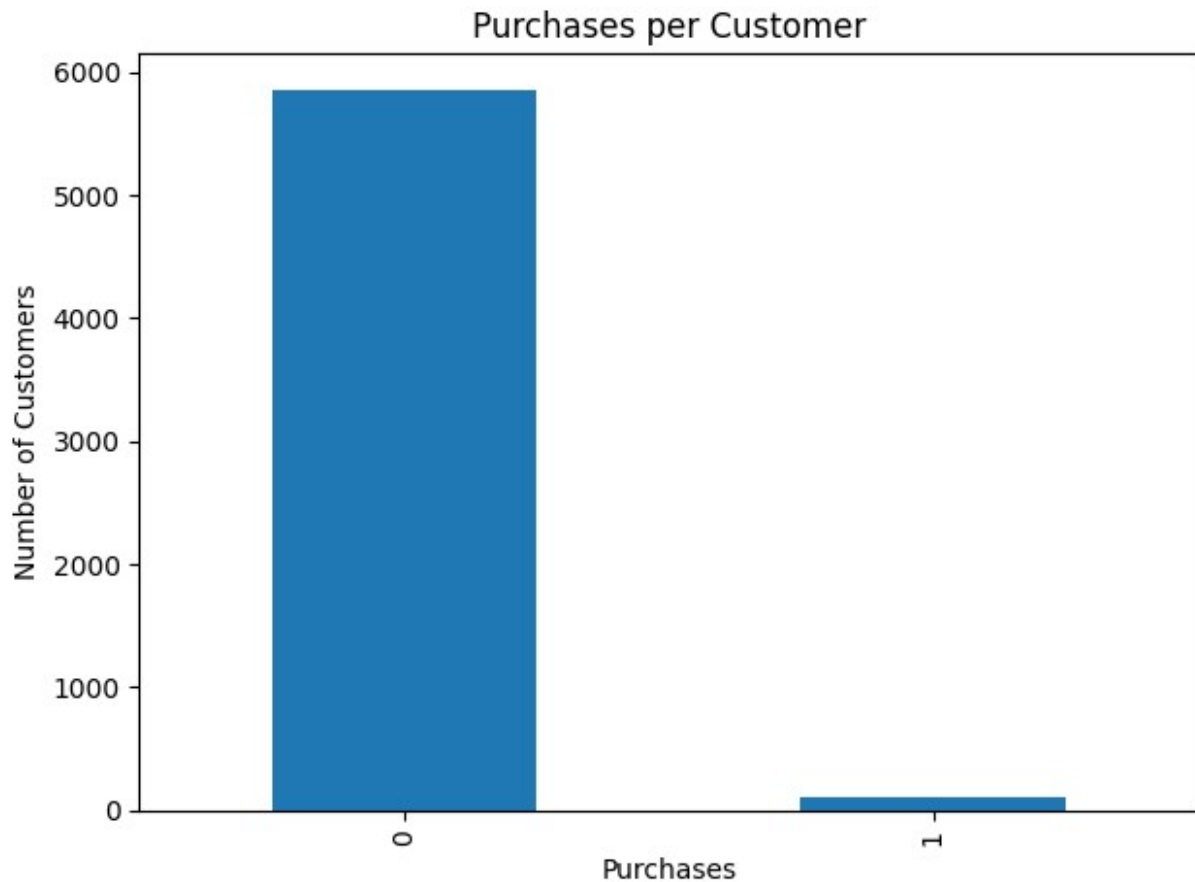
	count	mean	std	min	25%	50%	75%
\total_events	5961.0	1.006207	0.086671	1.0	1.0	1.0	1.0
views	5961.0	0.966449	0.213350	0.0	1.0	1.0	1.0
carts	5961.0	0.022144	0.147164	0.0	0.0	0.0	0.0
purchases	5961.0	0.017614	0.131557	0.0	0.0	0.0	0.0
sessions	5961.0	1.005200	0.080725	1.0	1.0	1.0	1.0
unique_products	5961.0	1.006207	0.086671	1.0	1.0	1.0	1.0
unique_categories	5961.0	1.003691	0.070851	1.0	1.0	1.0	1.0
unique_brands	5961.0	0.862607	0.354377	0.0	1.0	1.0	1.0
total_spend	5961.0	5.833830	68.505116	0.0	0.0	0.0	0.0
avg_purchase_value	5961.0	5.833830	68.505116	0.0	0.0	0.0	0.0
active_days	5961.0	1.004697	0.077577	1.0	1.0	1.0	1.0

recency_days	5961.0	15.946821	8.670305	1.0	9.0	16.0	23.0
cart_rate	5961.0	0.022032	0.146673	0.0	0.0	0.0	0.0
purchase_rate	5961.0	0.017614	0.131557	0.0	0.0	0.0	0.0

	max
total_events	4.00
views	4.00
carts	1.00
purchases	1.00
sessions	4.00
unique_products	4.00
unique_categories	4.00
unique_brands	2.00
total_spend	1747.79
avg_purchase_value	1747.79
active_days	4.00
recency_days	31.00
cart_rate	1.00
purchase_rate	1.00



```
customer["purchases"].value_counts().sort_index().plot(kind="bar",
title="Purchases per Customer")
plt.xlabel("Purchases")
plt.ylabel("Number of Customers")
plt.tight_layout()
plt.show()
```



5. Model Preparation

K-Means uses distance, so features are standardized.

Several count/value variables are highly right-skewed. We use `log1p` on those variables before scaling. This reduces the dominance of extreme values while preserving zeros.

We do **not** use customer IDs as model features.

```
model_features = [
    "views", "carts", "purchases", "sessions",
    "unique_products", "unique_categories", "unique_brands",
    "total_spend", "active_days", "recency_days"
]

X = customer[model_features].copy()

log_features = [
    "views", "carts", "purchases", "sessions",
    "unique_products", "unique_categories", "unique_brands",
    "total_spend", "active_days"
]
```

```

for col in log_features:
    X[col] = np.log1p(X[col])

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("Model matrix:", X_scaled.shape)

```

Model matrix: (5961, 10)

6. Select the Number of Clusters

We test multiple K values using:

- Silhouette Score — higher is better
- Calinski-Harabasz Score — higher is generally better
- Davies-Bouldin Score — lower is better

The final choice must consider both statistical quality and business interpretability.

```

results = []

for k in range(2, 8):
    model = KMeans(n_clusters=k, random_state=42, n_init=20)
    labels = model.fit_predict(X_scaled)

    results.append({
        "k": k,
        "inertia": model.inertia_,
        "silhouette": silhouette_score(X_scaled, labels),
        "calinski_harabasz": calinski_harabasz_score(X_scaled,
labels),
        "davies_bouldin": davies_bouldin_score(X_scaled, labels)
    })

metrics = pd.DataFrame(results)
display(metrics)

```

```

{"summary":{"\n  \"name\": \"metrics\", \n  \"rows\": 6, \n  \"fields\": [\n    {\n      \"column\": \"k\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 1, \n        \"min\": 2, \n        \"max\": 7, \n        \"num_unique_values\": 6, \n        \"samples\": [\n          2, \n          3, \n          7\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }\n    }, \n    {\n      \"column\": \"inertia\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 13168.747286653634, \n        \"min\": 6163.595267524811, \n        \"max\": 40788.04210866401, \n        \"num_unique_values\": 6, \n        \"samples\": [\n          40788.04210866401, \n

```

```

26681.14475789659,\n          6163.595267524811\n          ],\n
\"semantic_type\": \"\", \n          \"description\": \"\"\n          }\n
n    },\n    {\n        \"column\": \"silhouette\", \n
\"properties\": {\n            \"dtype\": \"number\", \n            \"std\":
0.1283087155482819, \n            \"min\": 0.6176275372697538, \n
\"max\": 0.9124983659994055, \n            \"num_unique_values\": 6, \n
\"samples\": [\n                0.9124983659994055, \n
0.8333854087158942, \n                0.6182056358206295\n            ], \n
\"semantic_type\": \"\", \n            \"description\": \"\"\n            }\n
n    },\n    {\n        \"column\": \"calinski_harabasz\", \n
\"properties\": {\n            \"dtype\": \"number\", \n            \"std\":
2241.8456912343413, \n            \"min\": 2749.8266961593017, \n
\"max\": 8604.82342867366, \n            \"num_unique_values\": 6, \n
\"samples\": [\n                2749.8266961593017, \n
3676.568627633516, \n                8604.82342867366\n            ], \n
\"semantic_type\": \"\", \n            \"description\": \"\"\n            }\n
n    },\n    {\n        \"column\": \"davies_bouldin\", \n
\"properties\": {\n            \"dtype\": \"number\", \n            \"std\":
0.02059405375586843, \n            \"min\": 0.4490326659480921, \n
\"max\": 0.5064845449334147, \n            \"num_unique_values\": 6, \n
\"samples\": [\n                0.48425895664987345, \n
0.46297877712485586, \n                0.49051329493180856\n            ], \n
\"semantic_type\": \"\", \n            \"description\": \"\"\n            }\n
n    }\n ]\n }\", \"type\": \"dataframe\", \"variable_name\": \"metrics\"}

```

```

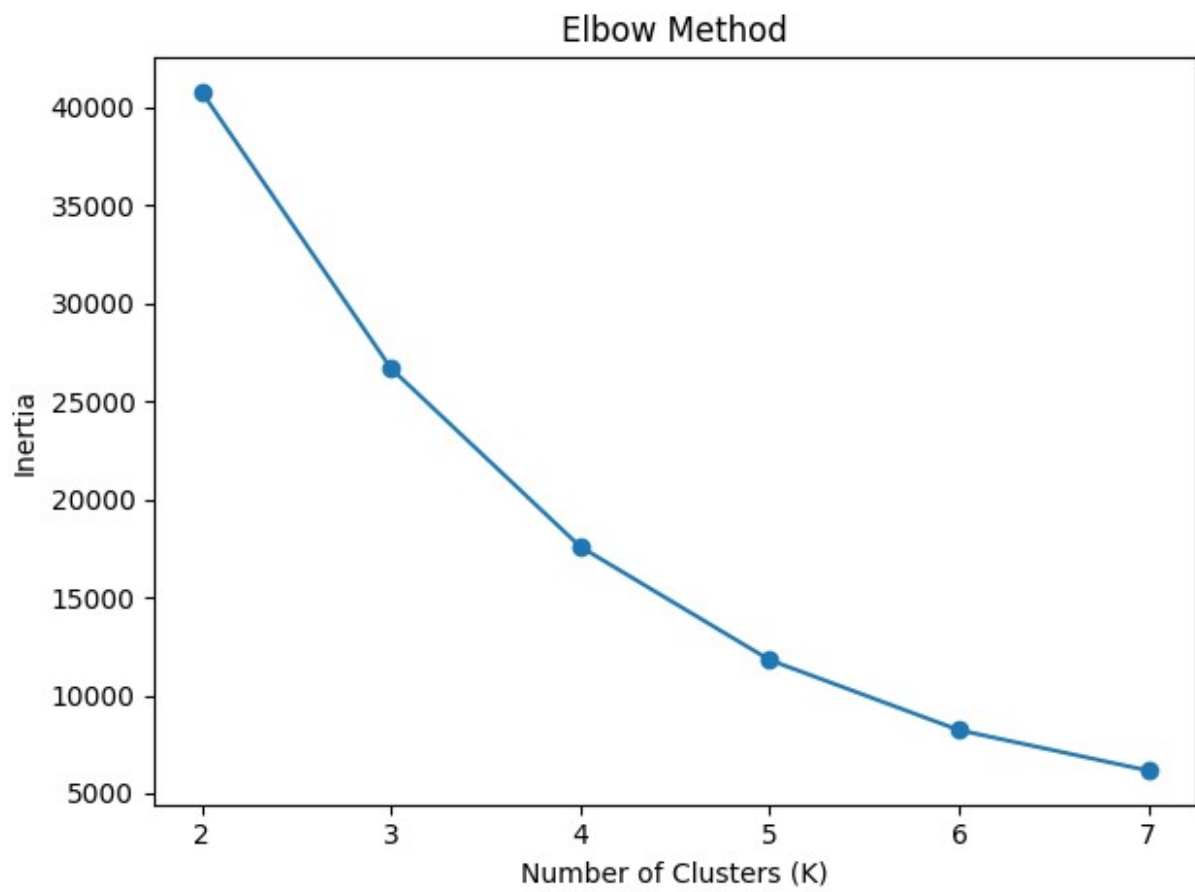
fig, ax = plt.subplots()
ax.plot(metrics[\"k\"], metrics[\"inertia\"], marker=\"o\")
ax.set_title(\"Elbow Method\")
ax.set_xlabel(\"Number of Clusters (K)\")
ax.set_ylabel(\"Inertia\")
plt.tight_layout()
plt.show()

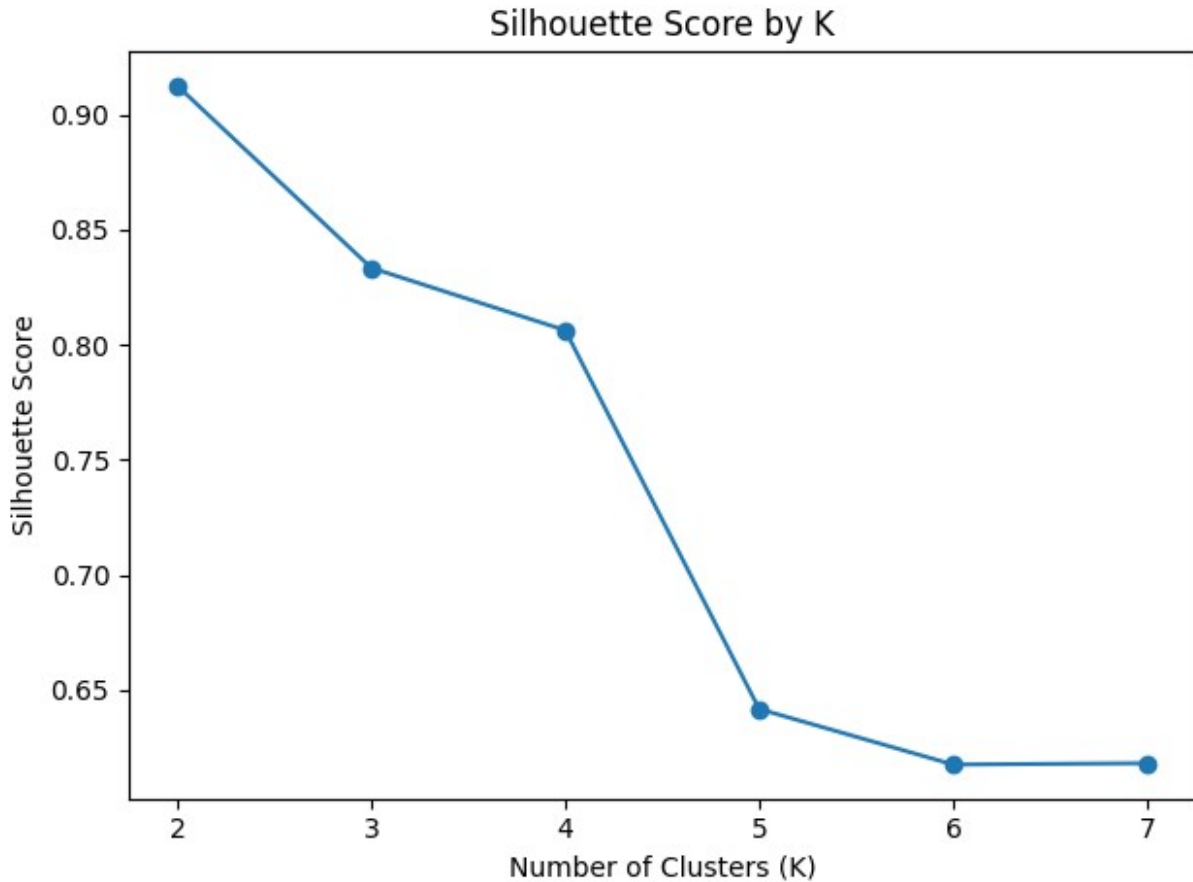
```

```

fig, ax = plt.subplots()
ax.plot(metrics[\"k\"], metrics[\"silhouette\"], marker=\"o\")
ax.set_title(\"Silhouette Score by K\")
ax.set_xlabel(\"Number of Clusters (K)\")
ax.set_ylabel(\"Silhouette Score\")
plt.tight_layout()
plt.show()

```





7. Final Segmentation Model

For this dataset, K=4 provides a useful business segmentation:

1. Casual Browsers
2. Purchasers
3. Highly Engaged Browsers
4. Cart Users / Cart Abandoners

Although K=2 has the highest silhouette score, it combines several commercially different behaviours. K=4 preserves clearer business actions while retaining good cluster separation.

This is a deliberate business-vs-statistical trade-off, not arbitrary model selection.

```
final_k = 4

kmeans = KMeans(n_clusters=final_k, random_state=42, n_init=20)
customer["cluster"] = kmeans.fit_predict(X_scaled)

print("Final silhouette score:", round(silhouette_score(X_scaled,
customer["cluster"]), 4))
```

```
print("\nCluster sizes:")
print(customer["cluster"].value_counts().sort_index())
```

Final silhouette score: 0.8063

Cluster sizes:

```
cluster
0      5695
1       105
2        30
3       131
Name: count, dtype: int64
```

8. Cluster Profiling

We profile every cluster using the original, interpretable feature values.

```
profile = customer.groupby("cluster")[model_features].agg(["count",
"mean", "median"]).round(2)
display(profile)
```

```
{"type": "dataframe", "variable_name": "profile"}
```

```
summary = customer.groupby("cluster")[model_features].mean().round(2)
display(summary)
```

```
{"summary": "{\n  \"name\": \"summary\",\n  \"rows\": 4,\n  \"fields\": [\n    {\n      \"column\": \"cluster\",\n      \"properties\": {\n        \"dtype\": \"int32\",\n        \"num_unique_values\": 4,\n        \"samples\": [\n          1,\n          3,\n          0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      {\n        \"column\": \"views\",\n        \"properties\": {\n          \"dtype\": \"number\",\n          \"std\": 0.9880409910524968,\n          \"min\": 0.0,\n          \"max\": 2.07,\n          \"num_unique_values\": 3,\n          \"samples\": [\n            1.0,\n            0.0,\n            2.07\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n        },\n      {\n        \"column\": \"carts\",\n        \"properties\": {\n          \"dtype\": \"number\",\n          \"std\": 0.49520197899443,\n          \"min\": 0.0,\n          \"max\": 1.0,\n          \"num_unique_values\": 3,\n          \"samples\": [\n            0.0,\n            0.03,\n            1.0\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n        },\n      {\n        \"column\": \"purchases\",\n        \"properties\": {\n          \"dtype\": \"number\",\n          \"std\": 0.5,\n          \"min\": 0.0,\n          \"max\": 1.0,\n          \"num_unique_values\": 2,\n          \"samples\": [\n            1.0,\n            0.0\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n        },\n      {\n        \"column\": \"sessions\",\n        \"properties\": {\n          \"dtype\": \"number\",\n          \"std\": 0.5149999999999999,\n          \"min\":
```

```

1.0,\n          \"max\": 2.03,\n          \"num_unique_values\": 2,\n\"samples\": [\n          2.03,\n          1.0\n          ],\n\"semantic_type\": \"\",\n\"description\": \"\"\n          }\n          },\n          {\n          \"column\": \"unique_products\",\n\"properties\": {\n          \"dtype\": \"number\",\n          \"std\": 0.55,\n          \"min\": 1.0,\n          \"max\": 2.1,\n          \"num_unique_values\": 2,\n          \"samples\": [\n          2.1,\n          1.0\n          ],\n          \"semantic_type\": \"\",\n\"description\": \"\"\n          }\n          },\n          {\n          \"column\": \"unique_categories\",\n\"properties\": {\n          \"dtype\": \"number\",\n          \"std\": 0.365,\n          \"min\": 1.0,\n          \"max\": 1.73,\n          \"num_unique_values\": 2,\n          \"samples\": [\n          1.73,\n          1.0\n          ],\n          \"semantic_type\": \"\",\n\"description\": \"\"\n          }\n          },\n          {\n          \"column\": \"unique_brands\",\n\"properties\": {\n          \"dtype\": \"number\",\n          \"std\": 0.3457720829293964,\n          \"min\": 0.85,\n          \"max\": 1.6,\n          \"num_unique_values\": 4,\n          \"samples\": [\n          0.92,\n          0.98\n          ],\n          \"semantic_type\": \"\",\n\"description\": \"\"\n          }\n          },\n          {\n          \"column\": \"total_spend\",\n\"properties\": {\n          \"dtype\": \"number\",\n          \"std\": 165.595,\n          \"min\": 0.0,\n          \"max\": 331.19,\n          \"num_unique_values\": 2,\n          \"samples\": [\n          331.19,\n          0.0\n          ],\n          \"semantic_type\": \"\",\n\"description\": \"\"\n          }\n          },\n          {\n          \"column\": \"active_days\",\n\"properties\": {\n          \"dtype\": \"number\",\n          \"std\": 0.46499999999999997,\n          \"min\": 1.0,\n          \"max\": 1.93,\n          \"num_unique_values\": 2,\n          \"samples\": [\n          1.93,\n          1.0\n          ],\n          \"semantic_type\": \"\",\n\"description\": \"\"\n          }\n          },\n          {\n          \"column\": \"recency_days\",\n\"properties\": {\n          \"dtype\": \"number\",\n          \"std\": 1.7951114171549354,\n          \"min\": 12.33,\n          \"max\": 16.09,\n          \"num_unique_values\": 4,\n          \"samples\": [\n          16.09,\n          15.64\n          ],\n          \"semantic_type\": \"\",\n\"description\": \"\"\n          }\n          }\n          ],\n\"type\": \"dataframe\", \"variable_name\": \"summary\"}

```

9. Business Persona Mapping

Cluster numbers have no inherent business meaning. We map them to personas based on observed behaviour.

The mapping should be validated against the profile table above.

```

# Determine cluster identities from behaviour.
cluster_means = customer.groupby("cluster")[model_features].mean()

buyer_cluster = cluster_means["purchases"].idxmax()

```

```

cart_cluster = cluster_means["carts"].idxmax()

remaining = [c for c in cluster_means.index if c not in
[buyer_cluster, cart_cluster]]

persona_map = {
    buyer_cluster: "Purchasers",
    cart_cluster: "Cart Users / Cart Abandoners"
}

customer["persona"] = customer["cluster"].map(persona_map)

print(persona_map)
display(customer["persona"].value_counts())

{np.int32(1): 'Purchasers', np.int32(3): 'Cart Users / Cart
Abandoners'}

persona
Cart Users / Cart Abandoners    131
Purchasers                      105
Name: count, dtype: int64

print(raw.columns.tolist())

['event_time', 'event_type', 'product_id', 'category_id',
'category_code', 'brand', 'price', 'user_id', 'user_session']

```

10. Persona Profiles

Casual Browsers

Low activity, mainly views, no cart and no purchase.

Action: product discovery, personalized recommendations, retargeting.

Purchasers

Customers with observed purchase activity and positive spend.

Action: cross-sell, upsell, loyalty and repeat-purchase campaigns.

Highly Engaged Browsers

Customers with multiple interactions but no observed purchase.

Action: personalized recommendations, conversion offers and retargeting.

Cart Users / Cart Abandoners

Customers who added an item to cart but did not purchase in the observed data.

Action: cart-recovery reminders, urgency messaging and targeted incentives.

These names are business interpretations of modelled clusters, not raw labels from the dataset.

```
persona_profile = customer.groupby("persona").agg(
    customers=("cluster", "size"),
    avg_events=("total_events", "mean"),
    avg_views=("views", "mean"),
    avg_carts=("carts", "mean"),
    avg_purchases=("purchases", "mean"),
    avg_sessions=("sessions", "mean"),
    avg_spend=("total_spend", "mean"),
    avg_recency_days=("recency_days", "mean")
).sort_values("customers", ascending=False).round(2)

display(persona_profile)

{"summary": "{\n  \"name\": \"persona_profile\",\n  \"rows\": 2,\n  \"fields\": [\n    {\n      \"column\": \"persona\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 2,\n        \"samples\": [\n          \"Purchasers\",\n          \"Cart Users / Cart Abandoners\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"avg_events\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.0,\n        \"min\": 1.0,\n        \"max\": 1.0,\n        \"num_unique_values\": 1,\n        \"samples\": [\n          1.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"avg_views\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.0,\n        \"min\": 0.0,\n        \"max\": 0.0,\n        \"num_unique_values\": 1,\n        \"samples\": [\n          0.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"avg_carts\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.7071067811865476,\n        \"min\": 0.0,\n        \"max\": 1.0,\n        \"num_unique_values\": 2,\n        \"samples\": [\n          0.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"avg_purchases\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
```

```
0.70711067811865476,\n      \"min\": 0.0,\n      \"max\": 1.0,\n      \"num_unique_values\": 2,\n      \"samples\": [\n        1.0\n      ],\n      \"semantic_type\": \"\",\n      \"description\": \"\"\n    },\n    {\n      \"column\": \"avg_sessions\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.0,\n        \"min\": 1.0,\n        \"max\": 1.0,\n        \"num_unique_values\": 1,\n        \"samples\": [\n          1.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"avg_spend\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 234.18669486117267,\n        \"min\": 0.0,\n        \"max\": 331.19,\n        \"num_unique_values\": 2,\n        \"samples\": [\n          331.19\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"avg_recency_days\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.31819805153394587,\n        \"min\": 15.64,\n        \"max\": 16.09,\n        \"num_unique_values\": 2,\n        \"samples\": [\n          16.09\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ],\n  \"type\": \"dataframe\", \"variable_name\": \"persona profile\"}
```

11. Visualize Personas

A 2D visualization is useful for communicating the segmentation to non-technical stakeholders. PCA is used only for visualization; the clustering model itself uses the full standardized feature space.

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_scaled)

plot_df = pd.DataFrame({
    "PC1": X_pca[:, 0],
    "PC2": X_pca[:, 1],
    "persona": customer["persona"].values
})

fig, ax = plt.subplots()
for persona in plot_df["persona"].unique():
    s = plot_df[plot_df["persona"] == persona]
    ax.scatter(s["PC1"], s["PC2"], label=persona, alpha=0.6)

ax.set_title("Customer Personas – PCA Projection")
ax.set_xlabel("Principal Component 1")
ax.set_ylabel("Principal Component 2")
ax.legend()
plt.tight_layout()
plt.show()
```



```
{"repr_error": "Out of range float values are not JSON compliant: nan", "type": "dataframe"}
```

14. Final Business Insights

1. The customer base is dominated by low-engagement browsing activity.
2. Purchase behaviour is comparatively rare in the observed month.
3. Cart users represent a high-intent group that can be targeted with cart-recovery campaigns.
4. Highly engaged browsers are an opportunity for conversion-focused personalization.
5. Purchasers can be targeted for retention, cross-sell and upsell.
6. The segmentation is a one-month behavioural snapshot; longer history and richer customer attributes would improve long-term persona stability.

Customer Segmentation & Persona Automation | Python, Pandas, Scikit-learn, K-Means

- Built an end-to-end customer segmentation pipeline on 6K+ e-commerce event records, transforming event-level data into customer-level behavioural features.
- Engineered engagement, purchase, session, product/category diversity, monetary and recency features while preserving the original source database.

- Evaluated K-Means solutions using inertia, Silhouette, Calinski-Harabasz and Davies-Bouldin metrics and selected a business-interpretable 4-segment solution.
- Developed actionable customer personas including Casual Browsers, Highly Engaged Browsers, Cart Users/Abandoners and Purchasers.
- Implemented an automated persona-assignment function using the trained scaler and clustering model.
- Translated customer clusters into targeted marketing recommendations for conversion, cart recovery, cross-sell and retention.

17. Limitations

- Only one month of data is available.
- Purchase frequency is limited because most customers have very few recorded events.
- Missing category and brand values exist.
- The personas should be treated as behavioural snapshots rather than permanent customer identities.
- A production solution would retrain periodically and monitor cluster drift.